

Day 3

7/31 금

어제 손대지 말라던 그 함수를 직접 짜는 날

01

화면에서 원하는 요소를 찾아 내용과 모양을 바꿀 수 있다

02

버튼을 누르면 무언가 일어나게 만들 수 있다

03

입력창에 쓴 값을 받아 화면에 반영할 수 있다

오늘의 흐름

HTML 과 CSS 부터 시작해서 화면을 움직이는 데까지

오전, 이론 09:00 - 12:50

■ 09:00	HTML 과 CSS 읽기
■ 10:00	요소 찾고 바꾸기
■ 11:00	반응하게 만들기
■ 12:00	입력 받기

오후, 실습 14:00 - 17:50

■ 14:00	미션 브리핑, 주제 선택
■ 15:00	개인 개발
■ 16:00	강사 순회, 개별 확인
■ 17:00	마무리

01 HTML 과 CSS 읽기

왜 지금 HTML 과 CSS 인가

쓰기 위해서가 아니라 읽고 고치기 위해서

- 오늘 배울 querySelector 의 괄호 안이 CSS 선택자
- 샵과 점이 뭔지 모르면 그냥 주문을 외우는 셈
- 다음 주에 각자 페이지를 처음부터 만듦. 그때 필요
- 외우지 않아도 됨. AI 가 가장 잘 쓰는 영역

목표가 아닌 것

태그를 외우고 빈 화면부터 손으로 짜기

오늘의 목표

받은 코드를 읽고, 어디를 고치면 무엇이 바뀌는지 아는 것

HTML 파일의 뼈대

모든 파일이 이 모양

```
1 <!DOCTYPE html>
2 <html lang="ko">
3 <head>
4   <meta charset="UTF-8">
5   <title>제목</title>
6   <style> ... </style>
7 </head>
8 <body>
9   <h1>보이는 제목</h1>
10  <script> ... </script>
11 </body>
12 </html>
```

화면에 안 보임
설정과 꾸밈

화면에 보임
여기를 만짐

태그는 감싸는 것

열고 내용 쓰고 닫기

```
<h1>오늘의 메뉴</h1>
```

여는 태그 여기서부터 제목이 시작	내용 화면에 실제로 보이는 글자	닫는 태그 슬래시가 붙음. 여기서 끝
-----------------------	----------------------	-------------------------

태그 안에 태그	<code><div>안녕</div></code>
----------	---

닫지 않는 태그	<code><input>
</code> 내용이 없어서 혼자 씀
----------	--

이것만 알면 오늘은 충분

외우지 말 것

<div>	덩어리를 묶는 상자. 줄이 바뀜	<button>	누를 수 있는 버튼
	글자 일부만 묶음. 줄이 안 바뀜	<input>	글자를 칠 수 있는 칸
<h1> ~ <h3>	제목. 숫자가 클수록 작아짐	 	목록과 목록 항목
<p>	문단	 <a>	그림과 링크

둘만 기억 **div** 는 덩어리, **span** 은 글자 일부. 오늘 코드의 대부분이 이 둘

속성, 태그에 붙는 정보

여는 태그 안에 씀

```
<input type="text" id="menu-input" placeholder="메뉴 추가">
```

id	이름표. 자바스크립트가 찾을 때 쓰는 것	오늘 가장 중요
class	묶음 이름. CSS 로 꾸밀 때 주로 씀	오늘 가장 중요
type	입력칸 종류. text, number, date 등	
placeholder	입력칸에 연하게 보이는 안내 글자	

id 와 class

오늘 이것만 확실히

■ id

한 페이지에 하나

```
<div id="result"></div>

// 찾을 때는 샵
querySelector("#result")
```

주민등록번호 같은 것. 딱 하나를 꼭 집을 때

■ class

여러 개 가능

```
<div class="card"></div>
<div class="card"></div>
// 찾을 때는 점
querySelectorAll(".card")
```

학과 같은 것. 같은 종류를 한꺼번에 다룰 때

외울 것 샵은 id 하나, 점은 class 여럿. 오늘 하루 종일 나옵니다

진짜 사이트를 뜯어봅시다

F12, Elements 탭

약 10분

아무 사이트에서 글자를 우클릭하고 검사를 눌러볼 것, 어제까지 쓰던 Console 옆 탭

CSS 문법

누구를, 무엇을, 어떻게

```
.card {  
  background: white;  
  padding: 20px;  
}
```

선택자

누구를 꾸밀지. 다음 장에서 자세히

속성

무엇을 바꿀지. 색, 크기, 여백

값

어떻게 바꿀지. 세미콜론으로 끝냄

파이썬과 비슷 중괄호 안에 이름 콜론 값. 딕셔너리와 모양이 같습니다

선택자는 세 곳에서 똑같이 쓰인다

한 번 익히면 세 군데에서 그대로 씀

파이썬 크롤링

```
soup.select(".card")
```

CSS

```
.card { background: white; }
```

오늘의 자바스크립트

```
document.querySelectorAll(".card")
```

h1

태그 이름

#result

id 하나

.card

class 여럿

#list .card

안쪽에 있는 것

이 정도만 알아도 화면이 바뀐다

외우지 말 것

color	글자 색
background	배경 색
font-size	글자 크기
font-weight	굵기. bold 또는 700

padding	테두리 안쪽 여백
margin	테두리 바깥쪽 여백
border	테두리. 굵기 종류 색
border-radius	모서리 둥글기

여백이 절반 화면이 답답해 보이면 색이 아니라 padding 을 늘려보세요

가로로 놓기

배치는 이거 하나만

그냥 두면 세로로 쌓임

```
<div class="row">
  <button>하나</button>
  <button>둘</button>
</div>
```

하나

둘

부모에 **flex** 를 주면 가로로

```
.row {
  display: flex;
  gap: 10px;
}
```

하나

둘

두 줄이면 끝 가로로 놓고 싶으면 감싼 쪽에 display flex 와 gap

CSS 는 어디에 쓰나

오늘은 첫 번째만

head 안 style 태그	<style> .card { ... } </style>	오늘 받을 파일
태그에 직접	<div style="color: red;">	급할 때만
별도 css 파일	<link rel="stylesheet" href="style.css">	규모가 커지면

이 과정에서는 파일 하나로 끝나게 첫 번째 방식만 씁니다

내 이름표 카드 만들기

```
// 1. div 안에 내 이름과 학과를 넣기  
// 2. 그 div 에 class 를 붙이고 CSS 로 꾸미기  
// 3. 배경색, 여백, 모서리 둥글기를 바꿔보기  
// 4. 이름과 학과를 가로로 나란히 놓기 (심화)
```

힌트 01

빈 파일부터 짜지 말 것. 나눠드린 파일의 표시된 곳만 채우면 됨

힌트 02

AI 에게 물어보셔도 됩니다. 대신 값을 바꿔가며 눈으로 확인할 것

남은 시간

15:00

02 요소 찾기

이틀 동안 우리는 콘솔에만 있었다

오늘부터 화면으로 나간다

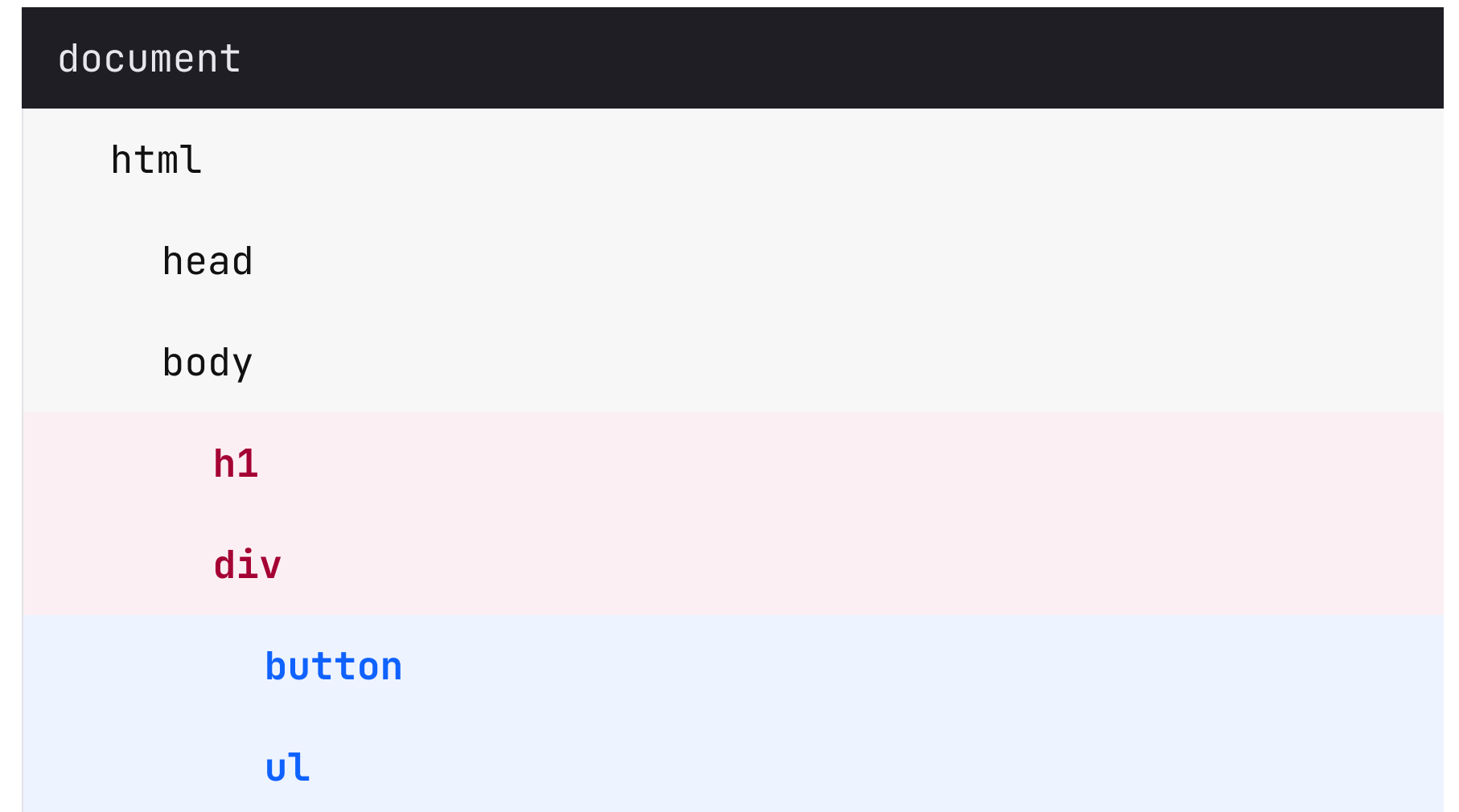
Day 1	변수와 조건문과 함수를 콘솔에서 실행	console.log
Day 2	30명 데이터를 걸러내고 정렬하고 합침	filter, sort, reduce
Day 3	그 결과를 실제 화면에 그리고, 버튼에 반응하게 만들기	DOM

어제 예고 손대지 말라고 한 render 함수, 오늘 직접 짜봅니다

DOM Document Object Model

브라우저가 HTML 을 읽어서 만든, 코드로 만질 수 있는 목록

- HTML 은 글자로 쓴 설계도
- 브라우저가 그걸 읽어서 요소 하나하나를 객체로 만들어 둬
- 그 객체 묶음이 DOM. 자바스크립트로 찾고 바꿀 수 있음
- 파일을 고치는 것이 아니라 브라우저 안의 객체를 고치는 것
- 그래서 새로고침하면 설계도대로 다시 만들어짐



크롤링에서 이미 해본 것

BeautifulSoup 과 DOM

■ Python, BeautifulSoup

읽기만

```
title = soup.select_one("h1")
print(title.text)

items = soup.select("h2")
print(len(items))

# 바꿀 수는 없음
```

■ JavaScript, DOM

읽고 쓰기

```
const title = document.querySelector("h1");
console.log(title.textContent);

const items = document.querySelectorAll("h2");
console.log(items.length);

title.textContent = "바뀜"; // 바꿀 수 있음
```

핵심 차이 select_one 이 querySelector, text 가 textContent. 그리고 바꿀 수도 있다

무엇으로 찾나

크롤링에서 쓴 그 선택자

```
1 // 태그 이름으로
2 document.querySelector("h1")
3
4 // id 로, 샵을 붙임
5 document.querySelector("#result")
6
7 // class 로, 점을 붙임
8 document.querySelectorAll(".card")
9
10 // 안쪽에 있는 것
11 document.querySelector("#list .card")
```

한 페이지에 하나만
오늘 가장 많이 씀

여러 개 가능

하나냐 여러 개냐

어제 find 와 filter 의 차이와 같다

■ querySelector

요소 하나

```
const title =  
  document.querySelector("h2");  
  
title.textContent // 바로 됨  
  
// 없으면 null
```

■ querySelectorAll

목록

```
const titles =  
  document.querySelectorAll("h2");  
  
titles.textContent // undefined  
titles[0].textContent // 됨  
  
// 없으면 빈 목록
```

주의 목록에 점을 찍으면 undefined. for of 로 돌거나 번호로 꺼내야 한다

오늘은 파일에 씁니다

practice.html

오전 내내

편집기와 브라우저를 나란히 띄워둘 것, 저장하고 새로고침을 계속 반복

내용 바꾸기

글자만 넣을 때와 태그까지 넣을 때

■ `textContent`

글자로만

```
box.textContent =  
  "안녕하세요";  
  
box.textContent =  
  "<b>굵게</b>";  
  
// 화면에 태그가 글자로 보임
```

■ `innerHTML`

태그까지

```
box.innerHTML =  
  "안녕하세요";  
  
box.innerHTML =  
  "<b>굵게</b>";  
  
// 실제로 굵게 나옴
```

어제 그 함수 어제 render 가 innerHTML 을 쓴 이유는 카드 태그를 함께 넣어야 했기 때문

practice.html 을 고쳐보기

```
// 1. 제목을 내 이름으로 바꾸기  
// 2. 문단이 몇 개인지 콘솔에 찍기  
// 3. 두 번째 문단만 바꾸기  
// 4. 빈 상자에 굵은 글씨를 넣기
```

힌트 01

두 번째는 querySelectorAll 로 받아서 번호로 꺼낼 것

힌트 02

4번은 태그를 넣어야 하므로 textContent 로는 안 됨

남은 시간

12:00

03 화면 바꾸기

모양 바꾸기, style

1일차에 콘솔로 해본 것

■ CSS에서는

하이픈

```
background-color: red;
font-size: 20px;
text-align: center;
border-radius: 8px;
```

■ 자바스크립트에서는

대문자

```
box.style.backgroundColor = "red";
box.style.fontSize = "20px";
box.style.textAlign = "center";
box.style.borderRadius = "8px";
```

규칙 하나 하이픈이 사라지고 그다음 글자가 대문자가 된다. 어제 lunchMenu 와 같은 규칙

style 을 여러 줄 쓰지 않는 방법

CSS 에 미리 써두고 class 만 붙이기

■ style 로 하나씩

네 줄

```
box.style.background = "#A50034";
box.style.color = "white";
box.style.padding = "12px";
box.style.borderRadius = "8px";

// 되돌리려면 또 네 줄
```

■ class 로 한 번에

한 줄

```
/* CSS 에 미리 써둬 */
.active { background: #A50034; ... }

// 자바스크립트에서는
box.classList.add("active");
box.classList.remove("active");
```

어제 그 버튼 어제 누른 버튼만 색이 칠해진 것도 classList 로 한 것

classList 네 가지

셋째가 오늘 가장 유용함

```
1 box.classList.add("active");
2 // 붙임. 이미 있으면 그대로
3
4 box.classList.remove("active");
5 // 뺌. 없어도 에러 안 남
6
7 box.classList.toggle("active");
8 // 있으면 빼고 없으면 넣음
9
10 box.classList.contains("active");
11 // 붙어 있는지 물어봄, true 나 false
```

켰다 켜는 데
가장 편함

없던 요소 만들기

만들고, 채우고, 붙이기

```
// 1. 만들기
const item = document.createElement("li");

// 2. 채우기
item.textContent = "김밥";
item.classList.add("food");

// 3. 붙이기
list.appendChild(item);
```

1 과 2 만 하면

만들어졌지만 화면에는 안 보임. 아직 어디에도 안 붙었기 때문

3 을 잊지 말 것

에러가 나지 않고 그냥 안 보임. 오늘 가장 흔한 실수

같은 결과, 두 가지 방법 둘 다 맞음

■ innerHTML

통째로 다시 그릴 때

```
list.innerHTML = foods
  .map(food =>
    `- ${food}</li>`
  )
  .join("");

// 짧음. 어제 render 가 이 방식

```

■ createElement

하나만 추가할 때

```
for (const food of foods) {
  const item =
    document.createElement("li");
  item.textContent = food;
  list.appendChild(item);
}

// 길지만 요소를 따로 다룰 수 있음
```

오늘은 왼쪽으로 갑니다. 어제 본 그 방식이고 코드가 짧습니다

어제 그 함수의 정체 dashboard.html 의 render

```
1 function render(list) {  
2   const listElement = document.querySelector("#list");  
3  
4   if (list.length === 0) {  
5     listElement.innerHTML = '...없습니다...';  
6     return;  
7   }  
8  
9   listElement.innerHTML = list.map(person => `  
10     <div class="card">  
11       <span>${person.name}</span>  
12     </div>  
13   `).join("");  
14 }
```

1교시에 배운
요소 찾기

어제 map 과
템플릿 문자열
거기에 innerHTML

전부 아는 것 새로 배운 건 `querySelector` 와 `innerHTML` 두 개뿐입니다

배열을 화면에 목록으로 그리기

```
const foods = ["김밥", "라면", "돈까스", "우동"];
```

```
// 1. 이 배열을 화면의 빈 상자에 목록으로 그리기
```

```
// 2. 글자 수가 3자 이상인 것만 그리기
```

힌트 01

18장 코드를 참고할 것. map 으로 태그를 만들고 join 으로 합침

힌트 02

2번은 앞에 filter 를 붙이면 됨. 글자 수는 length

남은 시간

15:00

04 반응하게 만들기

이벤트, 나중에 실행될 코드

지금까지 쓴 코드는 파일을 열 때 한 번 실행되고 끝났다

- 지금까지는 위에서 아래로 한 번 흐르고 끝
- 이벤트는 언제 실행될지 모르는 코드를 미리 맡겨두는 것
- 브라우저에게 이 버튼이 눌리면 이 함수를 실행해 달라고 부탁
- 눌릴 수도 있고 안 눌릴 수도 있음. 브라우저가 기다려 줌
- 여기서부터 프로그램이 프로그램처럼 느껴짐

Day 1 과 Day 2

파일을 열면 코드가 한 번 돌고 끝. 화면이 그대로 멈춘 상태

아래부터가 오늘

Day 3

사용자가 무언가 할 때마다 코드가 다시 실행됨

이틀 동안 쓴 방식과 오늘 배우는 방식

■ onclick, 지금까지

HTML 에 씀

```
←!— HTML →  
<button onclick="go()">눌러</button>  
  
// 자바스크립트  
function go() {  
  console.log("눌렀다");  
}
```

■ addEventListener, 오늘부터

JS 에만 씀

```
←!— HTML, 깨끗함 -->  
<button id="go-button">눌러</button>  
  
// 자바스크립트  
const button = document.querySelector("#go-button");  
button.addEventListener("click", () => {  
  console.log("눌렀다");  
});
```

왜 바꾸나 HTML 과 자바스크립트가 섞이지 않고, 한 요소에 여러 개를 붙일 수 있다

세 부분으로 읽기

누구에게, 무슨 일이, 무엇을

```
button.addEventListener("click", () => { ... });
```

누구에게

querySelector 로 찾아둔 요소. 1교시에 배운 것

무슨 일이 생기면

click, input, change 같은 이름. 따옴표로 감쌌

무엇을 할지

실행할 함수. 어제 map 안에 쓴 화살표 함수와 같은 모양

읽는 법 이 버튼에, 클릭이 일어나면, 이 함수를 실행해 달라

괄호 하나로 갈린다

오늘 가장 많이 나는 실수

```
1 function go() {  
2   console.log("눌렀다");  
3 }  
4  
5 button.addEventListener("click", go);  
6 // 함수 자체를 말김. 누를 때 실행됨  
7  
8 button.addEventListener("click", go());  
9 // 지금 실행해서 그 결과를 말김  
10 // 열자마자 한 번 찍히고, 눌러도 아무 일 없음
```

괄호 없음, 맞음

괄호 있음, 틀림
에러는 안 남

버튼이 여러 개일 때

하나씩 붙이지 말고 돌면서 붙이기

■ 하나씩 붙이면

버튼 수만큼

```
button1.addEventListener("click", ...);
button2.addEventListener("click", ...);
button3.addEventListener("click", ...);
```

// 버튼이 여섯 개면 여섯 줄

■ 돌면서 붙이면

몇 개든 같음

```
const buttons =
  document.querySelectorAll(".filter-button");

for (const button of buttons) {
  button.addEventListener("click", () => {
    console.log(button.textContent);
  });
}
```

어제 그 버튼들 어제 관심분야 버튼 여섯 개가 각각 동작한 것도 이 구조

이벤트 이름

오늘 쓸 것만

click

눌렀을 때. 오늘 가장 많이 씀

input

입력창에 글자를 칠 때마다. 4교시에 씀

change

선택 상자에서 고른 값이 바뀔 때

keydown

키를 누를 때. 오후 미션 중 하나에서 씀

외우지 말 것 이름이 수십 개 있습니다. 필요할 때 찾아 쓰면 됩니다

오늘 만날 에러 3

첫 번째가 압도적으로 많음

TypeError: Cannot read properties of null	원인 요소를 못 찾았음. 오타이거나, 코드가 요소보다 먼저 실행됨	해결 선택자 철자 확인. script 가 body 맨 아래에 있는지 확인
눌러도 아무 일이 없음	원인 함수 이름 뒤에 괄호를 붙였음	해결 24장 참고. 괄호를 떼거나 화살표 함수로 감쌀 것
만들었는데 화면에 안 보임	원인 createElement 만 하고 appendChild 를 안 했음	해결 16장 참고. 만들고 채우고 붙이기 세 단계

버튼을 살려보기

```
// 1. 버튼을 누르면 콘솔에 글자가 찍히게  
// 2. 버튼을 누르면 제목이 바뀌게  
// 3. 버튼을 누를 때마다 어두운 배경이 켜졌다 꺼지게
```

힌트 01

3번은 `classList.toggle`. CSS 에 `dark` 클래스가 미리 있음

힌트 02

눌러도 반응이 없으면 24장 괄호 문제를 확인할 것

남은 시간

15:00

05 입력 받기

입력값 읽기

파이썬 input 과 무엇이 다른가

■ Python

멈춰서 기다림

```
name = input("이름: ")

# 여기서 프로그램이 멈춤
# 사람이 칠 때까지 기다림

print(name)
```

■ JavaScript

안 기다림

```
const input =
  document.querySelector("#name-input");

// 버튼을 누를 때 읽음
button.addEventListener("click", () => {
  console.log(input.value);
});
```

핵심 차이 기다리지 않는다. 버튼이 눌린 그 순간에 value 를 읽는다

value 는 항상 글자다

숫자를 넣어도 글자로 온다

```
1 // 입력창에 3 을 쳤을 때
2 input.value           // "3"   글자
3 typeof input.value    // "string"
4
5 input.value + 1        // "31"  글자가 이어붙음
6
7 Number(input.value) + 1 // 4   숫자로 바꾼 뒤
8
9 // 비어 있는지 확인
10 if (input.value === "") return;
```

더하기가 아니라
이어붙이기가 됨

Number 로 감싸면 됨

오늘의 결론, 다시 그리기

화면을 조금씩 고치지 말고, 데이터를 고친 뒤 통째로 다시 그린다

1 사용자가 버튼을 누름

addEventListener

2 데이터를 고침. 배열에 넣거나 빼거나 값을 바꿈

push, filter

3 화면을 통째로 다시 그림

render

```
const todos = [];           // 데이터

addButton.addEventListener("click", () => {
  todos.push(input.value);    // 데이터를 고치고
  input.value = "";
  render();                  // 다시 그린다
});
```

할 일 목록, 직접 만들기

1일차에 AI 가 만들다 무너진 그것

```
const todos = [];  
const input = document.querySelector("#todo-input");  
const list = document.querySelector("#todo-list");  
  
function render() {  
  list.innerHTML = todos  
    .map(todo => `<li>${todo}</li>`)  
    .join("");  
}  
  
addButton.addEventListener("click", () => {  
  if (input.value === "") return;  
  todos.push(input.value);  
  input.value = "";  
  render();  
});
```

열여섯 줄 AI가 백 줄 넘게 준 그 기능이 오늘 배운 것만으로 열여섯 줄

1일차에 AI가 준 코드는 왜 그렇게 길었을까

방금 우리는 같은 기능을 열어섯 줄로 썼음. 각자 한 줄로 메모

할 일 목록 만들기

// 1. 입력창에 쓰고 버튼을 누르면 목록에 추가되게
// 2. 빈 칸으로 누르면 추가되지 않게
// 3. 지금 몇 개인지 위에 표시하기 (심화)

힌트 01

33장 코드를 그대로 옮겨 쓰셔도 됩니다

힌트 02

3번은 render 안에서 todos.length 를 함께 써주면 됨

남은 시간

18:00

셋 중 하나 골라 만들기

점심 메뉴 룰렛	반응속도 테스트	성향 테스트
버튼을 누르면 메뉴 하나가 무작위로 뽑힘 메뉴를 직접 추가하고 지울 수 있게	화면이 바뀌는 순간 얼마나 빨리 누르는지 측정 기록을 모아 최고 기록과 평균 표시	질문에 답하면 결과가 나오는 테스트 진행 상황과 결과 화면을 나눠 보이기
배열, 무작위, 입력값	시간 재기, 배열, 평균	객체, 조건문, 화면 전환

고르는 기준 난이도 차이 없음. 만들고 싶은 것으로. 5분 안에 정할 것

셋 다 구조가 같습니다

주제만 다르고 만드는 순서는 동일

1단계	데이터를 배열이나 객체로 준비하기	Day 2
2단계	화면에 그리는 render 함수 만들기	innerHTML
3단계	버튼에 이벤트를 붙이기	addEventListener
4단계	데이터를 고친 뒤 다시 그리기	render 재호출

전원 목표

3단계까지. 4단계 심화는 여유가 있을 때

방금 만든 것도 새로고침하면 사라진다

1일차에 남긴 그 질문. 답은 다음 주 화요일에

오늘 배운 것

- 01 요소를 찾아 내용과 모양을 바꿈
- 02 버튼을 누르면 무언가 일어나게 만듦
- 03 입력창에 쓴 값을 받아 화면에 반영함

다음 주 월요일 인터넷에서 데이터를 받아옵니다. 진짜 날씨를요